Curvature-Guided Dense Training for Post-Training N:M Pruning

Abstract

We introduce Joint, a dense-training objective designed for a later structured pruning operation. Joint estimates the second-order reconstruction cost of deleting two weights together in each N:M group. It uses a training-side activation Gram matrix to evaluate admissible deletion pairs, then forms a differentiable soft minimum of their weight-deletion and perturbation costs. Adding this signal to sharpness-aware minimization encourages a dense model to admit a low-cost structured deletion without fixing a mask during training. After the final checkpoint is frozen, an independent SparseGPT implementation performs dynamic mask selection and native OBS-style error compensation. This separates preparation of the dense weights from the subsequent one-shot pruner. Initial results suggest that the training signal is worth investigating, while a matched-initialization comparison remains necessary to establish its contribution.

I. Introduction

N:M pruning retains a fixed number of weights in each small group and thereby imposes a discrete support decision on a trained network. A dense model can be accurate before pruning but fragile to this decision. SparseGPT addresses the problem after training by selecting a mask and compensating the removed weights with an activation-based second-order approximation [1]. Its ability to act on an already trained model is valuable. We ask a complementary question: what if dense training itself made the future deletion less costly, while the final pruner remained unchanged?

Sharpness-aware minimization (SAM) trains with a local adversarial weight perturbation [2]. This promotes a form of robustness to continuous nearby changes, but the later N:M deletion is finite and restricted to a particular support. The SAM direction does not explicitly score which two coordinates in a group could be jointly removed. Moreover, a pairwise deletion need not have the sum of two independent single-weight costs: its effect depends on the interaction between those input coordinates. This distinction motivates a training signal aligned with the structured decision made after training.

We construct a differentiable proxy for the minimum two-coordinate deletion cost in each 2:4 group. It uses a damped layer-input Gram matrix, evaluates the six possible deletion pairs, and smoothly favors a low-cost pair. The proxy is added to the adversarial SAM objective. The network is never masked during dense training. After the final epoch, we apply the same frozen, audited SparseGPT adapter used for the reference endpoint, including its dynamic mask selection and OBS-style error compensation. The training proxy does not prescribe SparseGPT's final mask.

Our preliminary results show a positive accuracy pattern for this pipeline, but their interpretation requires restraint. The historical reference and the new training cohort have different, unverified initial weight identities. Consequently, the present comparison tests feasibility and motivates a matched study; it does not isolate a causal gain or establish a new method ranking. The scientific contribution at this stage is the training-to-pruning objective, its exact implementation path, and a clearly bounded initial evaluation.

II. Related Work

A. Sharpness-aware and compression-aware training

SAM constructs a local weight perturbation from the current gradient and updates using the loss at the perturbed weights [2]. ASAM changes the neighborhood geometry to account for parameter rescaling [3], and SSAM sparsifies the perturbation [4]. CrAM brings a compression operator into the training problem [5]. Our approach retains dense SAM optimization and adds an activation-derived structured deletion proxy. The discrete N:M mask is still selected only after training.

B. One-shot selection and compensation

Magnitude pruning removes small weights, while Wanda incorporates input activation norms into its score [6, 7]. SparseGPT selects weights with calibration statistics and propagates pruning error to surviving weights [1]; OBC

develops related second-order compression tools [8]. ROSE modifies SparseGPT's processing order [9]. These methods clarify why an immediate post-pruning endpoint must specify whether weight compensation has already occurred. Our training objective is separate from these final pruning algorithms and is evaluated primarily with SparseGPT's native compensated endpoint.

III. Method

A. Reconstruction geometry and a joint deletion

For an eligible convolutional layer ℓ, unfold training-side input activations into columns X_ℓ and form

$$H_\ell = \frac{2}{P} X_\ell X_\ell^\top, \qquad \widetilde{H}_\ell = H_\ell + \delta_\ell I, \qquad Q_\ell = \widetilde{H}_\ell^{-1}. \tag{1}$$

This is the reconstruction Gram matrix used by the training surrogate, not the exact Hessian of classification cross-entropy. The damping rule is fixed in advance, and the resulting factors are held constant between scheduled refreshes.

Within one output row, a 2:4 group g has six admissible deletion pairs D. For a pair, let w_D denote the two original dense weights and $Q_{\ell,DD}$ the corresponding 2×2 principal submatrix. The local quadratic cost is

$$C^w_{\ell,g,D} = \tfrac{1}{2} w_D^\top (Q_{\ell,DD})^{-1} w_D. \tag{2}$$

The off-diagonal terms account for the interaction between the two deleted coordinates. We also score deletion of the current SAM perturbation ϵ_D with the same quadratic form,

$$C^\epsilon_{\ell,g,D} = \tfrac{1}{2} \epsilon_D^\top (Q_{\ell,DD})^{-1} \epsilon_D. \tag{3}$$

The perturbation term is a training probe, not an executed pruning displacement. It connects the continuous neighborhood explored by SAM to the later support decision without asserting that they are identical.

B. Differentiable training signal

Let $r_\ell > 0$ be a frozen train-side reference cost for layer ℓ and $T > 0$ a fixed temperature. The implemented smooth minimum is

$$R_{\ell,g} = -T \log \left[\frac{1}{6} \sum_{D \in \mathcal{D}_g} \exp\left(-\frac{C^w_{\ell,g,D} + C^\epsilon_{\ell,g,D}}{T r_\ell} \right) \right]. \tag{4}$$

We average this quantity over eligible groups. The $1/6$ factor fixes the baseline of the soft minimum; it does not change which candidate is cheapest. With first-pass gradient $g_1 = \nabla_w L_B(w)$ and SAM perturbation $\epsilon = \rho g_1 / \|g_1\|_2$, the parameter update uses

$$L_{\text{train}} = L_B(w + \epsilon) + \lambda_p R(w, \epsilon; Q). \tag{5}$$

The first-pass gradient is detached when constructing ϵ for this regularizer. The optimizer combines the adversarial cross-entropy gradient with $\lambda_p \nabla_w R$. The perturbed weights are restored before the optimizer step, and the adversarial forward does not update batch-normalization running buffers. No mask, recovery training, or final-test feedback enters this dense-training objective.

C. Schedule and frozen readout

Training runs for a fixed 200 epochs. The first five epochs follow the underlying SAM objective; the pairwise signal is enabled afterward and its training-side factors are periodically refreshed. At the final checkpoint, a separate audited SparseGPT adapter constructs its own activation statistics, dynamically chooses the 2:4 mask, and applies sequential OBS-style compensation. Thus an improvement at this readout would concern the trained dense parent under the same final algorithm, rather than a change to SparseGPT's mask formula. The currently running sparse-in-time guidance schedule is not included in the completed results of this draft.

Algorithm 1 Dense training with a pairwise pruning proxy

Require: Training data, SAM model, fixed training and refresh schedule

for each dense-training epoch **do**
  if a training-side Gram refresh is scheduled **then**
    Cache layer matrices and the six pair factors in Eq. (2)
  end if
  for each mini-batch **do**
    Construct the SAM probe and adversarial cross-entropy
    if pairwise guidance is active **then**
      Add the differentiable cost in Eq. (4)
    end if
    Restore dense weights and update the model
  end for
end for
Freeze the final dense checkpoint
Apply the independent native SparseGPT selection and compensation
Evaluate the fixed sparse checkpoint once

IV. Experiments

A. Evaluation protocol

We use the official CIFAR-100 training set for dense optimization and calibration, a 100-class ResNet-18, and 2:4 sparsity on the frozen set of eligible internal convolutions. The test set is accessed only for a fixed final evaluation after each checkpoint is frozen. Table 1 reports immediate post-pruning accuracy, before any additional recovery epochs. It combines the completed historical three-run reference with the available two-run Joint study. Different run counts and initialization cohorts are shown explicitly; the table is not a matched causal ablation.

Table 1: CIFAR-100 immediate post-pruning accuracy (mean $\pm$ sample SD, %). Historical methods use three runs; Joint uses two and consists of Joint dense training followed by native SparseGPT. SparseGPT and ROSE already include second-order compensation; Magnitude and Wanda are mask-only. No row includes recovery.

Method	Accuracy
Magnitude	55.54 ± 1.36
Wanda	56.48 ± 3.83
SparseGPT	77.26 ± 0.42
ROSE	77.82 ± 0.19
Joint	78.53 ± 0.13

B. Reading the result

The Joint-to-SparseGPT endpoint has a higher mean than the historical SAM-to-SparseGPT endpoint. This pattern is compatible with the hypothesis that a dense model can be prepared for later second-order deletion. It is not yet proof of that hypothesis: the cohorts do not share verified identical initial weights, the number of runs differs, and a within-cohort SAM control has not been trained under the new initialization. Comparisons against Wanda and ROSE describe complete pipelines rather than isolating the training objective, because their immediate pruning mechanisms differ. In particular, the high immediate accuracy of SparseGPT and ROSE includes native compensation; it must not be interpreted as mask-only support quality.

V. Discussion

The method exposes a training-to-pruning connection: instead of predicting a hard mask during training, it encourages the dense model to admit a low-cost structured deletion under a frozen reconstruction proxy. The current evaluation is deliberately preliminary. A matched-initialization SAM control is the decisive next experiment for accuracy attribution. The ongoing sparse-in-time guidance experiment is not included in the completed results.

VI. Conclusion

We introduced a differentiable pairwise second-order deletion cost into dense SAM training and evaluated the resulting model with an unchanged, audited SparseGPT-style final pruner. Initial CIFAR-100 results are encouraging but cross-cohort. A matched comparison is required to establish whether the training signal, rather than initialization or another cohort difference, explains the observed accuracy pattern.

References

[1] Elias Frantar and Dan Alistarh. SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 10323–10337. PMLR, 2023. URL https://proceedings.mlr.press/v202/frantar23a.html.

[2] Pierre Foret, Ariel Kleiner, Hossein Mobahi, and Behnam Neyshabur. Sharpness-Aware Minimization for Efficiently Improving Generalization. In *International Conference on Learning Representations*, 2021. doi: 10.48550/arxiv.2010.01412. URL https://research.google/pubs/sharpness-aware-minimization-for-efficiently-improving-generalization/.

[3] Jungmin Kwon, Jeongseop Kim, Hyunseo Park, and In Kwon Choi. ASAM: Adaptive Sharpness-Aware Minimization for Scale-Invariant Learning of Deep Neural Networks. In *Proceedings of the 38th International Conference on Machine Learning*, 2021. doi: 10.48550/arxiv.2102.11600. URL https://proceedings.mlr.press/v139/kwon21b.html.

[4] Peng Mi, Li Shen, Tianhe Ren, Yiyi Zhou, Xiaoshuai Sun, Rongrong Ji, and Dacheng Tao. Make Sharpness-Aware Minimization Stronger: A Sparsified Perturbation Approach. In *Advances in Neural Information Processing Systems*, 2022. doi: 10.48550/arxiv.2210.05177. URL https://arxiv.org/abs/2210.05177.

[5] Alexandra Peste, Adrian Vladu, Eldar Kurtic, Christoph H. Lampert, and Dan Alistarh. CrAM: A Compression-Aware Minimizer. In *International Conference on Learning Representations*, 2023. doi: 10.48550/arxiv.2207.14200. URL https://arxiv.org/abs/2207.14200.

[6] Song Han, Jeff Pool, John Tran, and William J. Dally. Learning both Weights and Connections for Efficient Neural Network. In *Advances in Neural Information Processing Systems*, volume 28, 2015. URL https://proceedings.neurips.cc/paper/2015/hash/ae0eb3eed39d2bcef4622b2499a05fe6-Abstract.html.

[7] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. A Simple and Effective Pruning Approach for Large Language Models. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=PxoFut3dWW.

[8] Elias Frantar, Sidak Pal Singh, and Dan Alistarh. Optimal Brain Compression: A Framework for Accurate Post-Training Quantization and Pruning. In *Advances in Neural Information Processing Systems*, 2022. doi: 10.48550/arxiv.2208.11580. URL https://arxiv.org/abs/2208.11580.

[9] Mingluo Su and Huan Wang. ROSE: Reordered SparseGPT for More Accurate One-Shot Large Language Models Pruning, 2026. URL https://arxiv.org/abs/2603.05878.